Patch Management — Study Guide

Patch management is the operational, organizational discipline that everything else in this series (Nessus, OpenVAS, CVSS, VPR) ultimately feeds into — finding vulnerabilities is only half the problem; systematically getting them *fixed* across an entire organization, safely and on schedule, is its own distinct skill set that SOC/vulnerability management roles are built around.

---

1. What Patch Management Actually Covers

Patch management is the end-to-end process of identifying, testing, deploying, and verifying software/firmware updates across an organization's entire IT estate — OS patches, application updates, firmware updates, and security-specific hotfixes.

```
Vulnerability Scanning (Nessus/OpenVAS)  → identifies WHAT needs patching
Patch Management      → the process of actually GETTING patched, safely, at scale
```

---

2. The Patch Management Lifecycle

```
1. Discovery/Inventory  → know every asset that exists (can't patch what you don't
know about)
2. Vulnerability Assessment → scan to identify missing patches (Nessus/OpenVAS territory)
3. Prioritization → CVSS + VPR/EPSS + business context (which systems are critical/exposed)
4. Testing → apply patches in a non-production environment first
5. Deployment → roll out to production, often in phases/rings
6. Verification → confirm the patch actually applied and the vuln is remediated
7. Reporting → document compliance status, remaining gaps
```

This cycle repeats continuously — patch management is never "done," it's an ongoing operational rhythm, typically organized around a recurring cadence (see Patch Tuesday below).

---

3. "Patch Tuesday" — The Industry Rhythm

Microsoft releases security updates on the **second Tuesday of every month** ("Patch Tuesday") — this single date drives an enormous amount of enterprise IT/security team activity, since most organizations' patch cycles are built around testing and deploying Microsoft's monthly release.

```
Patch Tuesday (2nd Tuesday) → Microsoft releases updates
    ↓
Security teams triage → which patches address actively expl
oited/critical CVEs?
    ↓
Test in staging environment → typically within days
    ↓
Phased production rollout → typically within 1-4 weeks, dep
ending on org policy
```

Other vendors (Adobe, Oracle) have historically aligned their own release schedules near Patch Tuesday for similar predictability reasons — worth knowing this term specifically, since it comes up constantly in SOC/IT operations conversation.

---

4. Patch Prioritization — Tying Back to CVSS/VPR/KEV

This is where your CVSS and VPR study directly plugs in:

```
Priority 1 (patch immediately, out-of-cycle if needed):
  - On CISA KEV list (actively exploited)
  - Critical CVSS + High VPR/EPSS
  - Internet-facing system

Priority 2 (patch in next scheduled cycle):
  - High/Critical CVSS, but no confirmed active exploitatio
n
  - Internal-only system with some compensating controls
```

```
Priority 3 (patch on normal schedule):
  - Medium/Low CVSS
  - Low real-world exploitation signal
  - Well-segmented/low-exposure system
```

A mature patch management program doesn't patch strictly in CVSS order — it patches in **risk order**, exactly the distinction your VPR guide covered in depth.

---

5. Patching Strategies — Rollout Approaches

Strategy	How It Works	Trade-off
Ring-based deployment	Patch a small test group first, then expand in rings (IT → pilot users → broad org)	Slower but catches issues before broad impact
Canary deployment	Similar concept, a small "canary" subset gets patches first	Same idea, different terminology (borrowed from DevOps)
Big bang deployment	Patch everything simultaneously	Fast, but any patch-induced issue affects everyone at once
Maintenance windows	Patches applied only during pre-scheduled downtime windows	Predictable, minimizes business disruption, but slower response

Why phased rollout matters so much: patches occasionally break things (a genuinely common real-world problem) — deploying to a small test ring first catches compatibility issues before they cascade organization-wide, which is exactly why "just patch everything immediately" isn't actually the mature approach despite seeming most secure on paper.

---

6. Patch Management Tools (What Organizations Actually Use)

Tool	Platform Focus
WSUS (Windows Server Update Services)	Microsoft-native, free, on-prem Windows patching
Microsoft Intune / SCCM (Configuration Manager)	Enterprise Windows/endpoint management, cloud + on-prem

Tool	Platform Focus
Ansible / Puppet / Chef	Cross-platform automation, patching as part of broader config management
ManageEngine Patch Manager Plus	Cross-platform, third-party app patching (not just OS)
Qualys / Tenable.io	Combines vulnerability scanning + patch orchestration in one platform
apt/yum/dnf automation (unattended-upgrades, cron)	Linux-native patching, often scripted

```
# Linux - checking/applying patches manually (the fundament
als underneath any automation tool)
sudo apt update && sudo apt list --upgradable
sudo apt upgrade -y

# Automating unattended security updates on Debian/Ubuntu
sudo apt install unattended-upgrades
sudo dpkg-reconfigure unattended-upgrades
```

7. Special Case — Zero-Day Patching

When a vulnerability is being actively exploited *before* a patch exists (a true zero-day), the normal cycle breaks down entirely:

```
Zero-day disclosed  → no patch available yet
    ↓
Compensating controls applied immediately (WAF rules, network segmentation,
disabling the vulnerable feature/service entirely if possible)
    ↓
Vendor releases emergency out-of-band patch
    ↓
Emergency deployment, bypassing the normal testing/ring rollout timeline
    ↓
Post-incident review
```

This is exactly the scenario where the CISA KEV catalog and active threat intelligence (from your VPR guide) become operationally critical — knowing a CVE is a confirmed zero-day in active use is what triggers this accelerated path instead of the normal monthly cycle.

8. Common Patch Management Challenges (Real-World Friction Points)

- **Legacy/unsupported systems** — can't patch what the vendor no longer supports (a genuinely common, frustrating real-world constraint, especially in industrial/healthcare environments)
- **Uptime requirements** — some systems (hospitals, manufacturing lines) can't tolerate patching downtime easily, requiring careful maintenance window planning
- **Patch-induced breakage** — a patch fixing one issue occasionally breaks unrelated functionality, which is exactly why testing rings exist
- **Asset visibility gaps** — you can't patch systems you don't know exist (shadow IT, forgotten servers) — ties directly back to your Attack Surface Management concept from the OSINT guide
- **Third-party/application patching** — OS patching is often well-automated; keeping every third-party application (browsers, PDF readers, plugins) current is a much messier, frequently under-managed problem in practice

---

9. Patch Compliance and Reporting (SOC/Governance Relevance)

Organizations typically track:

```
Patch compliance rate     → % of assets fully patched withi
n SLA
Mean time to patch (MTTP) → average time from patch release
to deployment
Critical vuln aging       → how long Critical/High findings
remain open
```

These metrics feed into compliance frameworks you'll encounter in more mature SOC/GRC-adjacent roles — **PCI-DSS**, **ISO 27001**, **CIS Benchmarks** — all of which have explicit patch timeline requirements (e.g., "Critical vulnerabilities must be remediated within 30 days" is a common PCI-DSS-style requirement).

---

10. Patch Management's Place in the Bigger Security Picture

```
Recon/Scanning tools you've studied (Nessus, OpenVAS, Nuclei)
    ↓ identify what's vulnerable
CVSS + VPR/EPSS + CISA KEV
    ↓ prioritize what matters most
Patch Management process
    ↓ actually fix it, safely, at scale
Re-scan to verify
    ↓ confirm the fix worked
Report compliance status
```

This is genuinely the full-circle closure of your entire vulnerability management study series — from finding issues (Nmap/NSE/Nessus) through understanding severity (CVSS/VPR) to the actual organizational discipline of fixing them (Patch Management).

---

11. Kill Chain / ATT&CK Mapping

Patch management is fundamentally a **defensive control**, not an offensive technique — but it's the direct mitigation for an enormous share of ATT&CK techniques under **Exploitation** (T1190, T1203, T1068) and **Initial Access**. Nearly every "Mitigations" section on an ATT&CK technique page references patching/updating software as a core defense.

Quick Reference Cheat Sheet

```
Lifecycle: Discovery → Assessment → Prioritization → Testin
g → Deployment → Verification → Reporting
Patch Tuesday: 2nd Tuesday of every month (Microsoft)
Prioritize by: CVSS + VPR/EPSS + CISA KEV + business contex
t, not CVSS alone
Rollout strategies: Ring-based, Canary, Big Bang, Maintenan
ce Windows
Key tools: WSUS, Intune/SCCM, Ansible, ManageEngine, Qualy
s/Tenable.io
Key metrics: Patch compliance rate, MTTP, Critical vuln agi
ng
```
